ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

GAZIPUR-1704, BANGLADESH

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

PROJECT REPORT

EEE 4706

Microcontroller Based System Design Lab

PROJECT TITLE:

8051 Microcontroller-Based Scientific Calculator with Keypad & LCD

SUBMITTED BY:

Section A1

Group - 02

Name	Student ID
Ahmed Fatin Jawad	210021107
Mashrur Saad Emad	210021139
Faisal Abrar Chowdhury	210021141
Ahmed Awil Mohamoud	210021149
Bello Aminu Ado	210021157

DATE OF SUBMISSION: **April 20, 2026; Monday**

Table of Contents

1. Objective **3**
2. Required Components **3**
3. Circuit Diagram **4**
3.1 Microcontroller (U2 - AT89C51) 4
3.2 LCD Display (LCD1 - LCM016L) 4
3.3 4x4 Matrix Keypad 4
3.4 Push Button Interface 5
4. Features **5**
4.1 Mandatory Features 5
4.2 Additional Features 5
5. Working Principle **6**
5.1 System Overview 6
5.2 Keypad Scanning Algorithm 6
5.3 Number Formation & Expression Storage 6
5.4 Infix-to-Postfix Conversion (Shunting-Yard Algorithm) 6
5.5 Postfix Evaluation 6
5.6 Result Display 6
5.7 Algorithm Flowchart 7
6. Assembly Code **8**
7. Hardware Implementation **25**
8. Problems Faced **26**
Problem 1: Jump Address Out of Range with SJMP/AJMP 26
Problem 2: Register Bank Conflicts in Subroutines 26
Problem 3: Incorrect Number Formation for Multi-Digit Operands 26
Problem 4: LCD Busy Flag Handling 26
9. Contribution Table **27**
10. Conclusion **28**

1. Objective

The objective of this project is to design and implement a functional scientific calculator using the 8051 microcontroller (AT89C51). The system accepts mathematical expressions via a 4x4 matrix keypad, evaluates them for correct operator precedence, and displays both the input expression and the computed result on a 16x2 LCM016L LCD display.

The calculator is designed to fulfill the following operational goals:

- Perform the four basic arithmetic operations: addition, subtraction, multiplication, and division on operands up to 3 digits.
- Evaluate expressions containing nested parentheses with correct operator priority.
- Compute integer powers (square and cube) for single-digit operands using dedicated push buttons.

With two additional features:

Feature 1: Determine whether a user-entered number is prime

Feature 2: Calculate the GCD (Greatest Common Divisor) and LCM (Least Common Multiple) of two user-entered numbers.

2. Required Components

The following hardware components are used to build the calculator circuit. The table below lists each component, its quantity, unit price, and total estimated cost.

Component	**Description**	**Qty**	**Unit Price (BDT)**	**Total (BDT)**
Microcontroller Board	Main processing unit running at 12 MHz	1	7,000	7,000
LCM016L LCD (16x2)	Display for expression and result	1	150	150
Push Buttons (25x)	Numeric and operator input (digits 0-9, +, -, *, /), POW (^), IS PRIME?, GCD/LCM, = (Equal), RESET	25	5	125
Capacitors 33pF (C2, C3)	Crystal stabilization capacitors	1 set of 5	8	8
Capacitor 10uF (C1)	Power-on reset capacitor	1 set of 5	10	10
Resistor 4.7kΩ (R1-R5)	Pull-up resistors for push buttons	2 set of 5	10	20
Resistor Network 10kΩ (RN1)	Pull-up for port 0	2 set of 5	10	20
10kΩ Potentiometer (RV1)	LCD contrast adjustment	1	25	25
Breadboard	Circuit mounting board	1	145	145
Connecting Wires	Inter-component wiring	2 set	55	110

Estimated Total Project Cost: BDT 7,613 (approx.)

3. Circuit Diagram

The circuit is designed in Proteus ISIS and centers around the AT89C51 8051 microcontroller. The complete schematic is shown below, followed by a description of each subsystem.

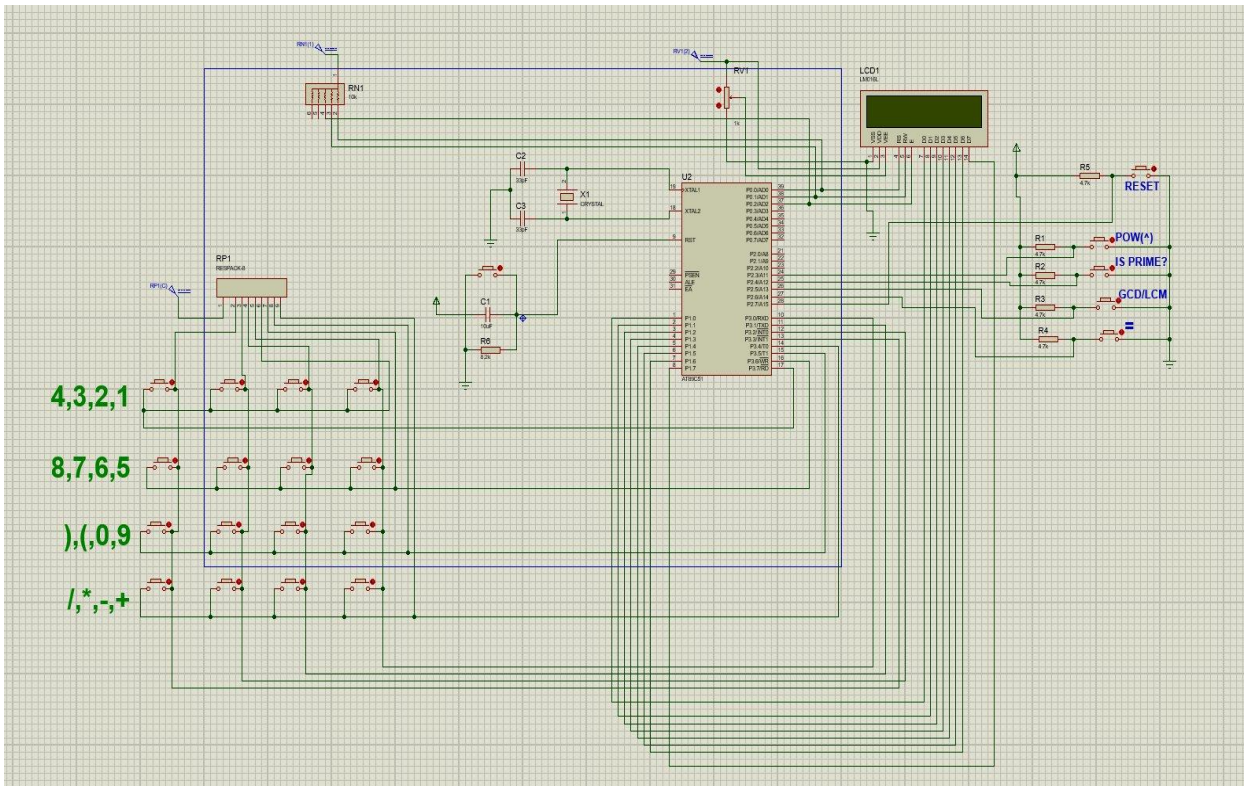


Figure 1: Proteus Circuit Schematic of the 8051 Calculator

3.1 Microcontroller (U2 - AT89C51)

The AT89C51 is the central processing unit. Port P0 (P0.0 to P0.7) is connected to the data bus of the LCD for sending 8-bit data and command bytes. Port P1 is used as the data bus to the LCD. Port P3 drives the 4x4 matrix keypad rows (P3.4-P3.7) and reads the column status. Port P2 (bits P2.3-P2.7) reads the five dedicated push buttons for POW(^), IS PRIME?, GCD/LCM, = (Equals), and RESET. The RST pin is connected to a standard power-on reset circuit (R6 + C1). The oscillator section uses an 11.0592 MHz crystal (X1) with 33pF capacitors (C2, C3).

3.2 LCD Display (LCD1 - LCM016L)

A 16x2 character LCD is used for user interaction. The RS, R/W, and Enable (E) control signals are driven by P0.0, P0.1, and P0.2 respectively. The 8-bit data lines (D0-D7) connect to Port P1. Contrast is adjusted via a 10kΩ potentiometer (RV1) connected to pin 3 (Vo). The LCD is operated in 8-bit mode and initialized with a 2-line, 5x7 character matrix configuration.

3.3 4x4 Matrix Keypad

The keypad provides numeric digits (0-9) and arithmetic operator keys (+, -, *, /, (,)). The 4 row lines are driven by the upper nibble of Port P3 (P3.4-P3.7), and the 4 column lines are read from the lower nibble. A resistor

network (RN1, 10kΩ SIP) provides pull-up for the column lines to ensure valid logic levels when no key is pressed. Key scanning uses a row-by-row approach: one row is pulled LOW at a time, and the column lines are checked for any LOW input to detect a keypress.

3.4 Push Button Interface

Five individual push buttons are connected to P2.3-P2.7 respectively. Each button is pulled HIGH via a 4.7kΩ resistor (R1-R5) and pulled LOW when pressed. The buttons serve as function triggers: POW (^) for exponentiation, IS PRIME? for prime checking, GCD/LCM for the GCD/LCM feature, = for expression evaluation, and RESET to restart the calculator.

4. Features

4.1 Mandatory Features

Basic Arithmetic Operations:

- Addition (+): Computes the sum of two operands.
- Subtraction (-): Computes the difference; supports negative results displayed with a leading minus sign.
- Multiplication (*): Computes the product of two operands.
- Division (/): Performs integer division of two operands.
- All four operations support operands of up to 3 decimal digits

Parenthesis and Operator Precedence:

The calculator correctly evaluates expressions like (3+5)*2 or (4+2)/(1+2) by implementing the Shunting-Yard algorithm to convert infix expressions to postfix (Reverse Polish Notation) before evaluation. Operator weights are: ^ = 3, * and / = 2, + and - = 1. Opening brackets push a context onto the operator stack; closing brackets pop and evaluate until the matching opening bracket.

Power Operation:

- The POW (^) push button inserts the caret operator into the expression.
- Supports x^2 (square) and x^3 (cube) for single-digit base values (0-9).
- Results are fetched from pre-computed look-up tables (SQUARE_DATA and CUBE_DATA) stored in code memory for efficiency.

4.2 Additional Features

Feature 1: Prime Number Checker

When the IS PRIME? button is pressed, the calculator switches to a dedicated prime-checking mode. The LCD prompts the user to enter a single number from the keypad. After input, the PRIME_CHECKER subroutine tests divisibility from 2 up to (n-1) using successive DIV operations. If no divisor is found, the Carry flag is set and the LCD displays "PRIME NUMBER"; otherwise it displays "NOT PRIME NUMBER".

Feature 2: GCD and LCM Calculator

The GCD/LCM push button triggers a two-number input mode. The user enters two numbers (A and B) separated by pressing the GCD/LCM button again. The GCD_LCM_CHECKER subroutine implements the Euclidean algorithm: it repeatedly computes the remainder of A divided by B and reassigns A=B, B=remainder, until the remainder is zero. The last non-zero divisor is the GCD. The LCM is then calculated as LCM = (A * B) / GCD. Both results are displayed on separate lines of the LCD.

5. Working Principle

5.1 System Overview

The calculator operates as an embedded system in a continuous input-evaluate-display loop. On power-on, the LCD is initialized and the system enters keypad scanning mode. The user builds an expression by pressing keys; each key press is debounced, identified, displayed on the LCD, and stored in an internal RAM buffer (starting at address 0x30H). When the = button is pressed, the stored expression is converted to postfix notation and evaluated.

5.2 Keypad Scanning Algorithm

The 4x4 keypad is scanned using a row-drive / column-sense method. The firmware drives all rows LOW and checks if any column line is LOW (KEYBOARD_STARTS). If no key is pressed, it monitors the five push button lines (P2.3-P2.7) continuously. When a keypress is detected, the firmware enters the row-identification phase (OVER1): it drives each row LOW one at a time while the remaining rows are HIGH, then reads the columns. The active row and column combination is used to index into ASCII look-up tables (KCODE0-KCODE3) to retrieve the character code of the pressed key.

5.3 Number Formation & Expression Storage

Digit keypresses build multi-digit numbers using the formula: current_value = (old_value x 10) + new_digit (implemented with MUL AB and ADD). The number is accumulated in register R4 and also written to the RAM buffer pointed to by R0. When an operator, bracket, or action button is pressed, number formation ends: R0 is incremented to the next address, R4 is cleared for the next number, and the operator symbol is stored in the following RAM cell. Register R2 tracks the total count of tokens stored.

5.4 Infix-to-Postfix Conversion (Shunting-Yard Algorithm)

On pressing =, the EQUAL_PUSH routine reads the token buffer and converts the infix expression to postfix using a software stack. Numeric tokens are sent directly to the output queue. Operators are compared using the PRECEDENCE function (which returns weight 3 for ^, 2 for * and /, 1 for + and -). Higher-precedence operators on the stack are popped to output before pushing the new operator. Opening brackets (are always pushed; a closing bracket) causes all operators to be popped until the matching (is found.

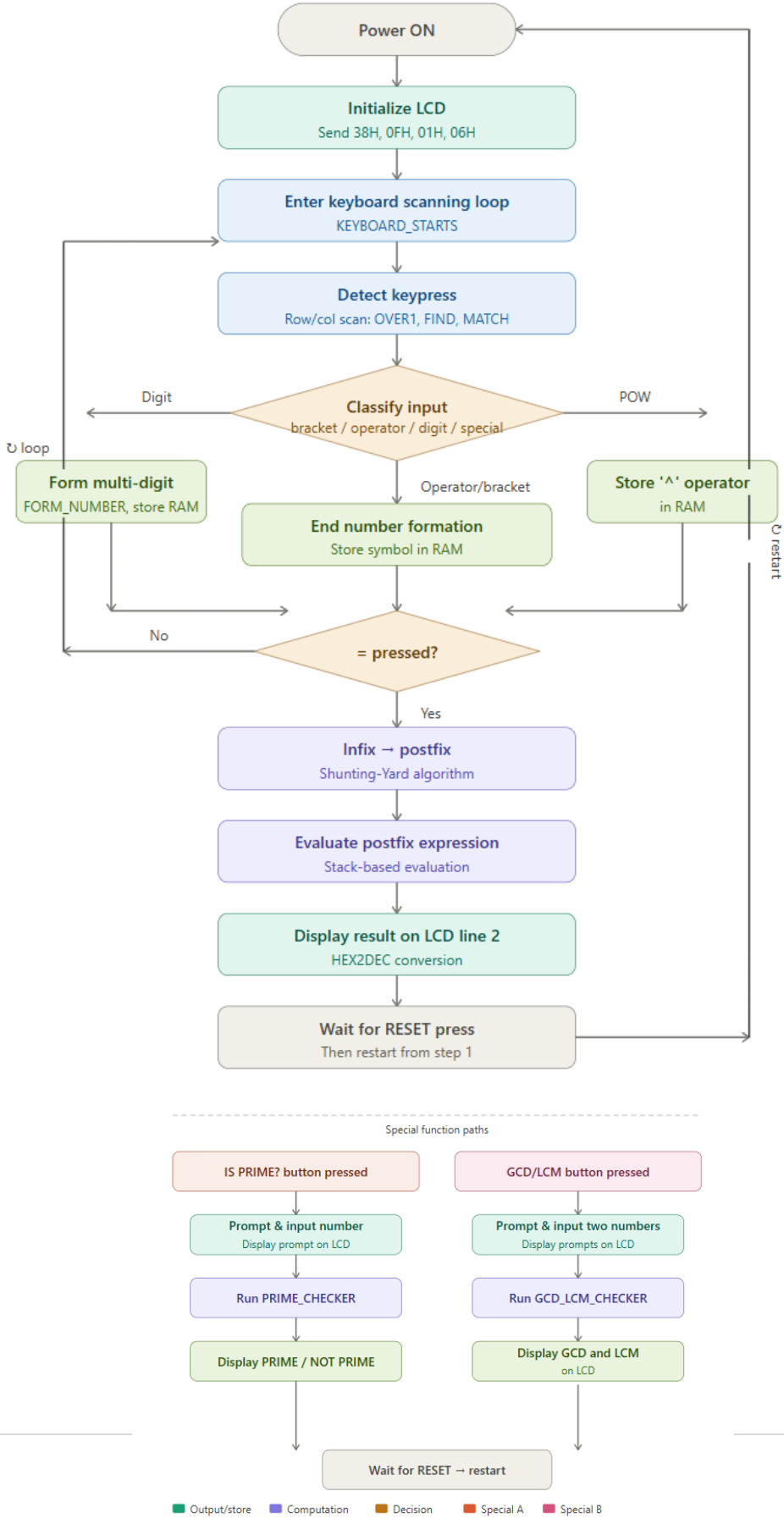
5.5 Postfix Evaluation

The postfix token queue is processed left-to-right. Numbers are pushed onto an evaluation stack. When an operator is encountered, two operands are popped, the corresponding arithmetic module is called (ADD_MODULE, SUB_MODULE, MUL_MODULE, DIV_MODULE, or POW_MODULE), and the result is pushed back. The final stack value is the expression result.

5.6 Result Display

The result (stored in R7) is checked for the Carry flag to detect negative values. If negative, 2's complement is applied before display. The HEX2DEC subroutine converts the binary result to three decimal digits (hundreds in R4, tens in R5, units in R6) using repeated division by 10. These digits are converted to ASCII and written to line 2 of the LCD.

5.7 Algorithm Flowchart



6. Assembly Code

The full assembly source code is written for the 8051 architecture (AT89C51) and assembled using the Mide-51 IDE. Key sections and subroutines are described and annotated below.

```
ORG  00H
   RS  EQU P0.0
   RW  EQU P0.1
   E  EQU P0.2
   KP_PORT EQU P3

  MAIN: MOV  SP, #70H
        MOV  PSW, #00H
        MOV R0,#30H ; POINTER FOR STORING NUMBERS,SYMBOLS AND BRACKETS
        MOV R2,#00H; COUNTER FOR HOW MANY MEMORY LOCATIONS WE ARE USING

LCD_IN:  MOV  A, #38H   ;init. LCD 2 lines, 5x7 matrix
   ACALL COMNWRT   ;call command subroutine
   ACALL  DELAY   ;give LCD some time
   MOV   A, #0FH   ;dispplay on, cursor on
   ACALL COMNWRT   ;call command subroutine
   ACALL  DELAY   ;give LCD some time
   MOV  A, #01    ;clear LCD
   ACALL COMNWRT   ;call command subroutine
   ACALL  DELAY   ;give LCD some time
   MOV  A, #06H   ;shift cursor right
   ACALL COMNWRT   ;call command subroutine
   ACALL  DELAY   ;give LCD some time
   MOV  A, #80H   ;cursor at line 1 postion 4
   ACALL COMNWRT   ;call command subroutine
   ACALL  DELAY   ;give LCD some time
   LJMP KEYBOARD_STARTS

  POW_PUSH_INTER:lJMP POW_PUSH ;PLACING THE INTERMEDIATE LOCATION HERE SO THAT
NOT ALL GETS EXECUTED AND ALSO WITHINN RANGE
  PRIME_PUSH_INTER: LJMP PRIME_PUSH
  GCD_PUSH_INTER: LJMP GCD_PUSH
  EQUAL_PUSH_INTER:LJMP EQUAL_PUSH
  RESET_PUSH_INTER:LJMP RESET_PUSH


;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;Keyboard
starts;;;;;;;;;;;;;;;;;;;
```

```
K2:JNB P2.3,POW_PUSH_INTER; IF POWER PUSH BUTTON IS PRESSED
   JNB P2.4,PRIME_PUSH_INTER; IF CHECK PRIME PUSH BUTTON IS PRESSED. JUMPING TO
INTERMEDIATE LOCATION AS ADDRESS WAS GOING OUT OF RANGE
   JNB P2.5,GCD_PUSH_INTER; IF GCD/LCM PUSH BUTTON IS PRESSED
   JNB P2.6,EQUAL_PUSH_INTER; IF EQUALS PUSH BUTTON IS PRESSED
   JNB P2.7,RESET_PUSH_INTER; IF RESET PUSH BUTTON IS PRESSED

   LCALL DELAY
   MOV A,KP_PORT
   ANL A,#00001111B
   CJNE A,#00001111B,OVER
   SJMP K2

OVER:LCALL DELAY
     MOV A,KP_PORT
   ANL A,#00001111B
   CJNE A,#00001111B,OVER1
   SJMP K2

OVER1:CLR KP_PORT.7
      SETB KP_PORT.6
      SETB KP_PORT.5
      SETB KP_PORT.4
      MOV A,KP_PORT
      ANL A,#00001111B
      CJNE A,#00001111B,ROW_0

      CLR KP_PORT.6
      SETB KP_PORT.7
      SETB KP_PORT.5
      SETB KP_PORT.4
      MOV A,KP_PORT
      ANL A,#00001111B
      CJNE A,#00001111B,ROW_1

      CLR KP_PORT.5
      SETB KP_PORT.7
      SETB KP_PORT.6
      SETB KP_PORT.4
      MOV A,KP_PORT
      ANL A,#00001111B
      CJNE A,#00001111B,ROW_2

      CLR KP_PORT.4
      SETB KP_PORT.7
      SETB KP_PORT.6
      SETB KP_PORT.5
      MOV A,KP_PORT
      ANL A,#00001111B
      CJNE A,#00001111B,ROW_3

ROW_0:MOV DPTR,#KCODE0
      SJMP FIND
```

```
ROW_1:MOV DPTR,#KCODE1
      SJMP FIND
ROW_2:MOV DPTR,#KCODE2
      SJMP FIND
ROW_3:MOV DPTR,#KCODE3
      SJMP FIND
FIND: RRC A
      JNC MATCH
      INC DPTR
      SJMP FIND

MATCH:  CLR PSW.3 ;MAKING SURE REGISTER BANK 0 KEEPS SELECTED
        CLR PSW.4
        CLR A
        MOVC A, @A+DPTR
        LCALL DATAWRT
        ;HERE WE HAVE TO FORM NUMBER UNTIL NO SYMBOLS HAVE BEEN PRESSED
        CLR A
        MOVC A,@A+DPTR ;AGAIN BRINGING THE NUMBER TO A IF GETS MODIFIED

        ;INITIALLY CHECKING IF THERE IS ANY BRACKETS. THEY WILL BE SENT DIRECTLY
TO THE MEMORY
        CJNE A,#'(',CHECK_1  ;COMPARE EACH AND EVERY SYMBOL AND JUMPS TO NEXT
KEY PRESS IF NOT EQUALS TO ANY NUMBER
        MOV @R0,A ; NUMBER IS NOT FORMED YET AS WE ARE STARTING 1ST BRACKET
        INC R0
        INC R2;KEEPING COUNT OF HOW MANY ITEMS ARE GOING INTO RAM LOCATIONS
        LJMP NEXT_KEY_PRESS
CHECK_1:CJNE A,#')',CHECK_2
        INC R0 ; GOING TO NEXT MEMORY AS NUMBER IS FORMED
        INC R2
        MOV R4,#00H; CLEARING THE OLD FORMING NUMBER
        MOV @R0,A ; ADING THE SYMBOL IN NEXT MEMOTY LOCATION
        INC R0
        INC R2;WE ARE NOT INCREMENTING IT AS WE ARE ASSUMING USER WILL PRESS A
SYMBOL OR EQUAL RIGHT AFTER ')' WHICH WILL END NUMBER FORMATION AND R2 WILL GET
INCREMENTED BY EITHER OPERATORS OR EQUAL SIGN
        LJMP NEXT_KEY_PRESS

CHECK_2:CJNE A,#'+',CHECK_3 ;ASSUMING NO WEIRD PEOPLE WILL PRESS A SYMBOL BEFORE
ANY NUMBER
        MOV A,R0; WE WILL CHECK THE ELEMENT BEFORE +. WHAT WAS IN THAT MEMORY.
NUMBER? OR ')'
        DEC A
        MOV R1,A
        MOV A,@R1;BRINGING THE PREVIOSU MEMORY CONTENT IN A
        CJNE A,#')',NUM_BEFORE_PLUS
        MOV @R0,#'+'
        INC R0
        INC R2
        LJMP NEXT_KEY_PRESS
NUM_BEFORE_PLUS:INC R0 ; GOING TO NEXT MEMORY AS NUMBER IS FORMED
                INC R2
```

```
                MOV R4,#00H; CLEARING THE OLD FORMING NUMBER
                MOV @R0,#'+' ; ADING THE SYMBOL IN NEXT MEMOTY LOCATION
                INC R0
                INC R2;KEEPING COUNT OF HOW MANY ITEMS ARE GOING INTO RAM
LOCATIONS
                LJMP NEXT_KEY_PRESS
CHECK_3:CJNE A,#'-',CHECK_4
        MOV A,R0; WE WILL CHECK THE ELEMENT BEFORE -. WHAT WAS IN THAT MEMORY.
NUMBER? OR ')'
        DEC A
        MOV R1,A
        MOV A,@R1;BRINGING THE PREVIOSU MEMORY CONTENT IN A
        CJNE A,#')',NUM_BEFORE_MINUS
        MOV @R0,#'-' ;AS THERRE WAS A ) BEFORE -. WE CAN JUST SEND THE'-' IN RAM
        INC R0
        INC R2
        LJMP NEXT_KEY_PRESS
NUM_BEFORE_MINUS:INC R0 ; GOING TO NEXT MEMORY
                 INC R2;KEEPING COUNT OF HOW MANY ITEMS ARE GOING INTO RAM
LOCATIONS
                 MOV R4,#00H; CLEARING THE OLD FORMING NUMBER
                 MOV @R0,#'-' ; ADING THE SYMBOL IN NEXT MEMOTY LOCATION
                 INC R0
                 INC R2;KEEPING COUNT OF HOW MANY ITEMS ARE GOING INTO RAM
LOCATIONS
                 LJMP NEXT_KEY_PRESS
CHECK_4:CJNE A,#'*',CHECK_5
        MOV A,R0; WE WILL CHECK THE ELEMENT BEFORE *. WHAT WAS IN THAT MEMORY.
NUMBER? OR ')'
        DEC A
        MOV R1,A
        MOV A,@R1;BRINGING THE PREVIOSU MEMORY CONTENT IN A
        CJNE A,#')',NUM_BEFORE_MUL
        MOV @R0,#'*' ;AS THERE WAS A ) BEFORE *. WE CAN JUST SEND THE '*' IN RAM
        INC R0
        INC R2
        LJMP NEXT_KEY_PRESS

NUM_BEFORE_MUL:INC R0 ; GOING TO NEXT MEMORY
               INC R2;KEEPING COUNT OF HOW MANY ITEMS ARE GOING INTO RAM
LOCATIONS
               MOV R4,#00H; CLEARING THE OLD FORMING NUMBER
               MOV @R0,#'*' ; ADING THE SYMBOL IN NEXT MEMOTY LOCATION
               INC R0
               INC R2;KEEPING COUNT OF HOW MANY ITEMS ARE GOING INTO RAM
LOCATIONS
               LJMP NEXT_KEY_PRESS
CHECK_5:CJNE A,#'/',FORM_NUMBER; IF NO SYMBOLS MATCHED THEN IT WAS A NUMBER
        MOV A,R0; WE WILL CHECK THE ELEMENT BEFORE /. WHAT WAS IN THAT MEMORY.
NUMBER? OR ')'
        DEC A
        MOV R1,A
        MOV A,@R1;BRINGING THE PREVIOSU MEMORY CONTENT IN A
```

```
        CJNE A,#')',NUM_BEFORE_DIV
        MOV @R0,#'/' ;AS THERE WAS A ) BEFORE /. WE CAN JUST SEND THE '/' IN RAM
        INC R0
        INC R2
        LJMP NEXT_KEY_PRESS

NUM_BEFORE_DIV:INC R0 ; GOING TO NEXT MEMORY
               INC R2;KEEPING COUNT OF HOW MANY ITEMS ARE GOING INTO RAM
LOCATIONS
               MOV R4,#00H; CLEARING THE OLD FORMING NUMBER
               MOV @R0,#'/' ; ADING THE SYMBOL IN NEXT MEMOTY LOCATION
               INC R0
               INC R2 ;KEEPING COUNT OF HOW MANY ITEMS ARE GOING INTO RAM
LOCATIONS
               LJMP NEXT_KEY_PRESS

FORM_NUMBER:CLR PSW.3; WE WILL USE THE R3,R4,R5 OF BANK 0 TO TEMPORARY STORE THE
DIGITS TO FORM NUMBER
            CLR PSW.4
            CLR C
            SUBB A,#30H ; CONVERTING TO HEX FROM ASCII
            MOV R3,A; R3 IS STROING THE NEW DIGIT

            MOV A,R4; BRINGING OLD VALUE IN ACCUMULATOR
            MOV B,#10; PREPARING FOR MUL
            MUL AB ; OUR RESULT WILL BE IN A AS LOWER BYTE AS WE ARE NOT ALLOWING
NUMBERS GREATER THAN 255
            ADD A,R3; ADDING THE NEW DIGIT TO FORM THE CURRENT NUMBER
            MOV R4,A; R4 WILL KEEP THE OLD NUMBER FOR NEXT ITERATION
            MOV @R0,A; STROING THE NUMBER ALSO DIRECTLY IN CURRENT MEMORY

NEXT_KEY_PRESS:LCALL DELAY
               LJMP K1
 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;KEYPAD
ENDS;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;PUSH BUTTON
STARTS;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
 POW_PUSH:LCALL DELAY;PREVENTING DEBOUNCING
          LCALL DELAY
          LCALL DELAY
          LCALL DELAY
          LCALL DELAY
          MOV A,#'^'
          LCALL DATAWRT
          LCALL DELAY
          MOV A,R0
          DEC A
          MOV R1,A
          MOV A,@R1
          CJNE A,#')',NUM_BEFORE_POW
          MOV @R0,#'^' ;AS THERE WAS A ) BEFORE ^. WE CAN JUST SEND THE '^' IN
RAM
```

```
          INC R0
          INC R2
          LJMP K1

NUM_BEFORE_POW:INC R0 ; GOING TO NEXT MEMORY AS NUMBER IS FORMED
               INC R2;KEEPING COUNT OF HOW MANY ITEMS ARE GOING INTO RAM
LOCATIONS
               MOV R4,#00H; CLEARING THE OLD FORMING NUMBER
               MOV @R0,#'^'; ALSO STORING THESE EXTRA SYMBOLS IN RAM LOCATIONS
               INC R0
               INC R2
               LJMP K1

PRIME_PUSH:MOV DPTR,#PRIME_PROMPT1
PRIME_MSG1:CLR A
           MOVC A, @A+DPTR
           JZ FINISH_PRIME1
           LCALL DATAWRT
           LCALL DELAY
           INC DPTR
           LJMP PRIME_MSG1

FINISH_PRIME1:MOV A,#0C0H
              LCALL COMNWRT
              LCALL DELAY

           MOV DPTR,#PRIME_PROMPT2
PRIME_MSG2:CLR A
           MOVC A, @A+DPTR
           JZ FINISH_PRIME2
           LCALL DATAWRT
           LCALL DELAY
           INC DPTR
           LJMP PRIME_MSG2
FINISH_PRIME2:MOV A,#'P' ; STROING P IN THE MEMORY TO INDICATE WE HAVE TO CHECK
IF PRIME OR NOT FOR NUMBERS AFTERWARDS
              MOV @R0,A; ALSO STORING THESE EXTRA SYMBOLS IN RAM LOCATIONS
               INC R0
               INC R2
              LJMP K1

EQUAL_PUSH:LCALL DELAY;PREVENTING DEBOUNCING
           LCALL DELAY
           LCALL DELAY
           LCALL DELAY
           LCALL DELAY
           LCALL DELAY
           MOV A,R0
           DEC A
           MOV R1,A
           MOV A,@R1
           CJNE A,#')',FINAL_NUM_FORMING
           LJMP START_CALCULATING
```

```
FINAL_NUM_FORMING:INC R2; NUMBER ALREADY FORMED. SO ADDING +1 TO THE COUNTER
                  MOV A,#'='
                  LCALL DATAWRT
                   LCALL DELAY
                  LJMP START_CALCULATING

GCD_PUSH:MOV DPTR,#GCD_PROMPT1
GCD_MSG1:CLR A
         MOVC A,@A+DPTR
         JZ FINISH_GCD1
         LCALL DATAWRT
         LCALL DELAY
         INC DPTR
         LJMP GCD_MSG1

FINISH_GCD1:MOV A,#0C0H
             LCALL COMNWRT
             LCALL DELAY

            MOV DPTR,#GCD_PROMPT2
GCD_MSG2:   CLR A
           MOVC A,@A+DPTR
           JZ FINISH_GCD2
           LCALL DATAWRT
           LCALL DELAY
           INC DPTR
           LJMP GCD_MSG2
FINISH_GCD2:MOV A,#'G' ; STROING G IN THE MEMORY TO INDICATE WE HAVE TO FIND GCD
& LCM OF 2 NEXT MEMORY LOCATION NUMBERS
            MOV @R0,A; ALSO STORING THESE EXTRA SYMBOLS IN RAM LOCATIONS
             INC R0
             INC R2
            LJMP K1

RESET_PUSH: MOV A,#01H
             ;LCALL COMNWRT
             ;LCALL DELAY
             ;LJMP K1 ;PROBABLY INTERRUPT WILL BE ADDED TO MASTER RESET FUNCTION
             LJMP MAIN
 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;PUSH BUTTON
ENDS;;;;;;;;;;;;;;;;;;;
```

```
                    MOV A,#01H ;CLEARING THE SCREEN. DISPLAYING AS PRIME NUMBER
                    LCALL COMNWRT
                    LCALL DELAY
                    MOV DPTR,#YES_PRIME
YES_PRIME_MSG:       CLR A
                    MOVC A,@A+DPTR
                    JZ FINISH_PRIME_PRINTING
                    LCALL DATAWRT
                    LCALL DELAY
                    INC DPTR
                     SJMP YES_PRIME_MSG

PRINT_NOT_PRIME: MOV A,#01H ;CLEARING THE SCREEN. DISPLAYING AS NOT A PRIME
NUMBER
                  LCALL COMNWRT
                  LCALL DELAY
                  MOV DPTR,#NO_PRIME
NO_PRIME_MSG:      CLR A
                  MOVC A, @A+DPTR
                  JZ FINISH_PRIME_PRINTING
                  LCALL DATAWRT
                  LCALL DELAY
                  INC DPTR
                  SJMP NO_PRIME_MSG

FINISH_PRIME_PRINTING: LJMP DONE_DISPLAY

CHECK_GCD_MODULE: CJNE A,#'G',STANDARD_ARITHMATIC
                    MOV A, 32H ; OUR DEFINED INPUT RULE WILL STORE THE 2 NUMBERS IN
32H AND 35 H RAM LOCATIONS
                    MOV B, 35H
                    LCALL GCD_LCM_CHECKER

                    MOV R0,A ;KEEPING THE GCD TEMPORARILY IN R0
                    MOV R1,B ;KEEPING THE LCM TEMPORARILY IN R1

                    MOV A,#01H ;CLEARING THE SCREEN. DISPLAYING LCM AND GCD
                    LCALL COMNWRT
                    LCALL DELAY

                    MOV DPTR,#GCD_ANS
GCD_ANS_PRINTING:  CLR A
                    MOVC A, @A+DPTR
                    JZ FINISH_GCD_PRINTING_PROMPT
                    LCALL DATAWRT
                    LCALL DELAY
                    INC DPTR
                     SJMP GCD_ANS_PRINTING

FINISH_GCD_PRINTING_PROMPT:MOV A,R0
                           LCALL HEX2DEC
                             ;CODES FOR DISPLAY WHERE MSB=R4 LSB=R6
```

```
                        MOV A,R4
                        ADD A,#30H
                        LCALL DATAWRT
                        LCALL DELAY

                       MOV A,R5
                       ADD A,#30H
                       LCALL DATAWRT
                       LCALL DELAY

                       MOV A,R6
                       ADD A,#30H
                       LCALL DATAWRT
                       LCALL DELAY

                        MOV A,#0C0H ;START PRINTING LCM IN NEXT LINE
                       LCALL COMNWRT
                       LCALL DELAY

                       MOV DPTR,#LCM_ANS
LCM_ANS_PRINTING:      CLR A
                       MOVC A,@A+DPTR
                       JZ FINISH_LCM_PRINTING_PROMPT
                       LCALL DATAWRT
                       LCALL DELAY
                       INC DPTR
                       SJMP LCM_ANS_PRINTING

FINISH_LCM_PRINTING_PROMPT:MOV A,R1
                        LCALL HEX2DEC
                          ;CODES FOR DISPLAY WHERE MSB=R4 LSB=R6
                        MOV A,R4
                        ADD A,#30H
                        LCALL DATAWRT
                        LCALL DELAY

                       MOV A,R5
                       ADD A,#30H
                       LCALL DATAWRT
                       LCALL DELAY

                       MOV A,R6
                       ADD A,#30H
                       LCALL DATAWRT
                       LCALL DELAY
                       LJMP DONE_DISPLAY ;DONE DISPLAYING GCD AND LCM


;;;;;;;DONE PRIME OR GCD/LCM OPERATION;;;;;;;;;;;;;;;;;;;;;;;;;

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;START CALCULATING
RESULTS;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
STANDARD_ARITHMATIC: MOV R0,#30H ;STARTING MEMORY OF ALREADY STORED ITEMS
```

```
                    MOV R1,#50H ;STARTING MEMORY OF POST FIX EXPRESSION
                    MOV SP,#70H
                    MOV R4,#00H ; COUNTER FOR POST FIX EXPRESSION ELEMENTS


CHECK_NEXT_INFIX: MOV A,@R0
                  CJNE A,#'(',CHECK_CLOSING_BRACKET
                   PUSH 0E0H
                   INC R0
                  LJMP NEXT_INFIX_CONDITION

CHECK_CLOSING_BRACKET: CJNE A,#')',CHECK_ADDITION_CONDITION
popTILL_FIRST_BRACKET: POP 0E0H
                       CJNE A,#'(',STORE_POPPED_SYMBOL
                       INC R0 ; IF ( FOUND THAN WE WILL DISCARD IT AND START
PARSING NEXT INFIX ITEM
                       LJMP NEXT_INFIX_CONDITION

STORE_POPPED_SYMBOL: MOV @R1,A ;SYMBOLS EXCEPT BRACKETS ARE GETTING STORED
                     INC R1
                     INC R4
                     SJMP popTILL_FIRST_BRACKET

CHECK_ADDITION_CONDITION:       CJNE A,#'+',CHECK_SUB_CONDITION
                                LJMP OPERATOR_CONDITION ;ALL OPERATORS WILL BE
DEALT SIMILARLLY
CHECK_SUB_CONDITION:            CJNE A,#'-',CHECK_MULTIPLICATION_CONDITION
                                LJMP OPERATOR_CONDITION; ALL OPERATORS ARE GOING
IN SAME LABEL
CHECK_MULTIPLICATION_CONDITION: CJNE A,#'*',CHECK_DIVISION_CONDITION
                                LJMP OPERATOR_CONDITION
CHECK_DIVISION_CONDITION:       CJNE A,#'/',CHECK_POW_CONDITION
                                LJMP OPERATOR_CONDITION
CHECK_POW_CONDITION:            CJNE A,#'^',DEFAULT_CONDITION
                                LJMP OPERATOR_CONDITION

OPERATOR_CONDITION:             MOV B,A ; [KEEPING FETCHED ITEM IN B TO PRESERVE]
WHILE_OPERATOR:                 MOV A,SP ; INITIALTING A FOR LOOP FOR OPERATOR
PRECEDENCE COMPARSION
                                CJNE A,#70H,CHECK_AND_CONDITION_FOR_WHILE;
CHECKING IF WE REACHED BOTTOM OF STACK
                                LJMP BREAK_WHILE_OPERATOR
CHECK_AND_CONDITION_FOR_WHILE: MOV A,B ; RESTORING OLD STORED ITEM
                                LCALL PRECEDENCE
                                MOV A,R5
                                MOV R6,A;KEEPING THE PRECEDENCE VALUE OF CURRENT
ITEM IN R6
                                MOV A,SP; TRYING TO GET THE TOP MOST ELEMENT OF
STACK
                                ;DEC A
                                SETB PSW.3
                                CLR PSW.4 ;USING REGISTER BANK 1 FOR POINTER
OPERATION
```

```
                                MOV R0,A;ADDRESS OF TOP MOST ELEMENT IN STACK
                                MOV A,@R0
                                CLR PSW.3 ; GOING BACK TO REGISTER BANK 0
                                LCALL PRECEDENCE;GETTIG PRECEDENCE OF TOP STACK
ELEMENT
                                MOV A,R5
                                MOV R7,A; STORING PRECEDENCE OF TOP STACK ELEMENT
IN R7

                                MOV A,R7 ; PREPARING FOR COMPARISON
                                CLR C
                                SUBB A,R6 ; CARRY WILL BE 1 IF CURRENT OPERATOR
HAS HIGHER PRIORITY THAN STACK
                                JC BREAK_WHILE_OPERATOR;BREAKING FROM THE WHILE
LOOP
                                POP 0E0H ; IF LAST STACK ELEMENT HAD HIGHER
PRECEDENCE
                                MOV @R1,A ; SENDING TO THE POST FIX ARRAY
                                INC R1
                                INC R4
                                LJMP WHILE_OPERATOR ; CONTINUING THE WHILE
OPERATION

BREAK_WHILE_OPERATOR:           MOV A,B ;BRINGING CURRENT ELEMENT IF IT HAD HIGHER
PRIORITY THAN STACK ELEMS
                                PUSH 0E0H
                                INC R0
                                LJMP NEXT_INFIX_CONDITION
DEFAULT_CONDITION:              MOV @R1,A ;IT WAS A OPERAND. SO NEED TO PROCESS
                                INC R1
                                INC R4
                                INC R0

NEXT_INFIX_CONDITION:             DJNZ R2,CHECK_NEXT_INFIX
                                SJMP CHECK_LEFTOVERS

CHECK_LEFTOVERS:                MOV A,SP ; NOW ANY REMAINIG ELEMENTS IN STACK WILL
DIRECTLY BE PUT INTO POST FIX ARRAY
                                CJNE A,#70H,EMPTY_STACK ;WILL START EMPTYING STACK
IF NOT EQUAL
                                SJMP DONE_POSTFIX; IF SP IS 70 THAN NOTHING STACK
AND WE ARE DONE

EMPTY_STACK:            POP 0E0H
                        MOV @R1,A
                        INC R1
                        INC R4
                        LJMP CHECK_LEFTOVERS

DONE_POSTFIX:
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;DONE POSTFIX EXPRESSION
FORMING;;;;;;;;;;;;;;;;;;;
```

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;START EVALUATING POSTFIX
EXPRESSION;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

MOV R1,#50H ;STARTING LOCATION OF POSTFIX EXPRESSION
;NO. OF POSTFIX ELEMENTS ARE IN R4
POST_EVALUATION: MOV A,@R1
                 CJNE A,#'+',CHECK_SUB_MODULE ;FIRST CHECKING CONDITIONS FOR EACH
OPERATORS
                 POP 0F0H ;TOP MOST STACK ELEMENT IN B
                 POP 0E0H; 2ND ELEMENT FROM TOP IS IN A
                 LCALL ADD_MODULE
                 PUSH 0E0H ; PUSHING THE RESULT OF OPERATION TO THE STACK
                 INC R1; GOING FOR NEXT ELEMENT IN POSTFLIX_ARRAY
                 LJMP NEXT_POSTFLIX_CONDITION

CHECK_SUB_MODULE:CJNE A,#'-',CHECK_MUL_MODULE
                 POP 0F0H ;TOP MOST STACK ELEMENT IN B
                 POP 0E0H; 2ND ELEMENT FROM TOP IS IN A
                 LCALL SUB_MODULE
                 PUSH 0E0H ; PUSHING THE RESULT OF OPERATION TO THE STACK
                 INC R1; GOING FOR NEXT ELEMENT IN POSTFLIX_ARRAY
                 LJMP NEXT_POSTFLIX_CONDITION

CHECK_MUL_MODULE:CJNE A,#'*',CHECK_DIV_MODULE
                 POP 0F0H ;TOP MOST STACK ELEMENT IN B
                 POP 0E0H; 2ND ELEMENT FROM TOP IS IN A
                 LCALL MUL_MODULE
                 PUSH 0E0H ; PUSHING THE RESULT OF OPERATION TO THE STACK
                 INC R1; GOING FOR NEXT ELEMENT IN POSTFLIX_ARRAY
                 LJMP NEXT_POSTFLIX_CONDITION

CHECK_DIV_MODULE:CJNE A,#'/',CHECK_POW_MODULE
                 POP 0F0H ;TOP MOST STACK ELEMENT IN B
                 POP 0E0H; 2ND ELEMENT FROM TOP IS IN A
                 LCALL DIV_MODULE
                 PUSH 0E0H ; PUSHING THE RESULT OF OPERATION TO THE STACK
                 INC R1; GOING FOR NEXT ELEMENT IN POSTFLIX_ARRAY
                 LJMP NEXT_POSTFLIX_CONDITION

CHECK_POW_MODULE:CJNE A,#'^',DEFAULT_NUMBERS
                 POP 0F0H ;TOP MOST STACK ELEMENT IN B
                 POP 0E0H; 2ND ELEMENT FROM TOP IS IN A
                 LCALL POW_MODULE ;WILL EVALUATE A TO THE POWER B
                 PUSH 0E0H ; PUSHING THE RESULT OF OPERATION TO THE STACK
                 INC R1; GOING FOR NEXT ELEMENT IN POSTFLIX_ARRAY
                 LJMP NEXT_POSTFLIX_CONDITION

DEFAULT_NUMBERS: PUSH 0E0H ;IF NUMBER FOUND THEN GETTING PUSHED INTO STACK
                 INC R1
                 SJMP NEXT_POSTFLIX_CONDITION
NEXT_POSTFLIX_CONDITION: DJNZ R4,POST_EVALUATION
                         POP 0E0H ; POPPING THE LAST ELEMENT FROM STACK WHICH IS
OUR RESULT. CURRENTLY IN A
```

```
                        SJMP DONE_POST_EVALUATION
DONE_POST_EVALUATION: MOV R7,A; KEEPING THE RESULT IN R7 SO THAT IT DOESN'T GET
LOST

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;DONE EVALUATING POSTFIX
EXPRESSION;;;;;;;;;;;;;;;;;;;
```

```
                ADD A,#30H
                LCALL DATAWRT
                LCALL DELAY

                MOV A,R6
                ADD A,#30H
                LCALL DATAWRT
                LCALL DELAY
                LJMP DONE_DISPLAY

DONE_DISPLAY:
INFINITE_LOOP:JNB P2.7,MAIN_INTER
              SJMP INFINITE_LOOP
MAIN_INTER:    LJMP MAIN
 ;;;
```

```
        CLR PSW.3
        CLR PSW.4
        RET


;PRECEDENCE FUNCTION ASSIGNS WEIGHT TO EACH OPERATOR TO COMARE PRIORITY
; OPERATOR IS IN A AND RETURNED WRIGHT WILL BE IN R5
PRECEDENCE: CJNE A,#'^',CHECK_DIVISION_PRECEDENCE
            MOV R5,#03H ;3 IS WEIGHT OF ^
            RET
CHECK_DIVISION_PRECEDENCE:CJNE A,#'/',CHECK_MULTIPLICATION_PRECEDENCE
                          MOV R5,#02H; 2 IS WEIGHT FOR BOTH DIV AND MUL
                          RET
CHECK_MULTIPLICATION_PRECEDENCE: CJNE A,#'*',CHECK_ADDITION_PRECEDENCE
                                 MOV R5,#02H
                                 RET
CHECK_ADDITION_PRECEDENCE: CJNE A,#'+',CHECK_SUB_PRECEDENCE
                           MOV R5,#01H; 1 IS THE WRIGHT OF ADD AND SUB
                           RET
CHECK_SUB_PRECEDENCE:      CJNE A,#'-',DEFAULT_PRECEDENCE
                           MOV R5,#01H
                           RET
DEFAULT_PRECEDENCE:     MOV R5,#00H
                        RET

;DIFFERENT ARITHAMTIC OPERATION MODULES:
ADD_MODULE: ADD A,B ; THE OPERANDS MUST BE IN A&B
            RET
SUB_MODULE: CLR C
            SUBB A,B ; THE OPERANDS MUST BE IN A&B
            RET
MUL_MODULE: MUL AB ; THE OPERANDS MUST BE IN A&B
            RET
DIV_MODULE: DIV AB ; THE OPERANDS MUST BE IN A&B
            RET
POW_MODULE:MOV R7,A; TEMPORARY STORING THE A VALUE TO CHECK THE POWER
           MOV A,B
           CJNE A,#02,CHECK_CUBE
           MOV DPTR,#SQUARE_DATA
           MOV A,R7 ; AGAIN RESTORING THE BASE VALUE
           MOVC A, @A+DPTR
           SJMP DONE_POW_EVALUATING
CHECK_CUBE:MOV DPTR,#CUBE_DATA
           MOV A,R7 ; AGAIN RESTORING THE BASE VALUE
           MOVC A, @A+DPTR
DONE_POW_EVALUATING: RET

;THIS FUNCTION WILL CHECK IF NUMBER IS PRIME OR NOT. INPUT MUST BE IN A
PRIME_CHECKER:     CLR PSW.3;SELECTING REGISTER BANK 2
                   SETB PSW.4
                   ;MOV R3,#0 ; COUNTER FOR DIVIDABLE NUMBERS
                   MOV R4,#2 ; STARTING DENOMINATOR
```

```
                MOV R2,A ; STORING THE ORIGINAL NUMBER AS IT WILL BE CHANGED
AFTER DIV OPERATION

AGAIN_PRIME_CHECK:MOV A,R2
                MOV B,R4
                DIV AB ; REMAINDER IS IN B

                INC R4; GOING TO THE NEXT DENOMINATOR

                MOV A,R2;MOVING THE ORIGINAL NUMBER IN A FOR COMPARISON USING
SUB
                CLR C
                SUBB A,R4
                JZ PRIME ;F4 IS NOW EQUALS TO THE NUMBER AND STILL NOTHING GOT
DIVIDED. SO A PRIME NUMBER

                MOV A,B ; COMPARING THE REMAINDER WITH 0
                CJNE A,#00H,AGAIN_PRIME_CHECK ;ADND IF EQUALS TO 0 THEN NOT
PRIME
                CLR C ; SETTING LOW THE FLAG FOR NOT PRIME ,AS IT GOT DIVIDED
                SJMP EXIT_PRIME_CHECKER

               PRIME:SETB C ; SETTING HIGH THE FLAG FOR PRIME

EXIT_PRIME_CHECKER:CLR PSW.4 ;GOING BACK TO DEFAULT REGISTER BANK 0
                 CLR PSW.3
                 RET

;THIS FUNCTION FINDS GCD AND LCM OF 2 NUMBERS STORED IN A AND B
GCD_LCM_CHECKER:CLR PSW.3;SELECTING REGISTER BANK 2
              SETB PSW.4
              MOV R6,A; TEMPORARY STORING
              MOV R7,B

              CLR C ;CHECKING IF A IS GREATER THAN B OR NOT. IF NOT THEN
SWAPPING A & B
              SUBB A,B
              JNC OKAY_FOR_GCD
              PUSH 16H ;SWAPING A & B
              PUSH 17H
              POP 16H
              POP 17H

OKAY_FOR_GCD:      MOV A,R6;BRINGING THE ORIGINAL NUMBERS
                 MOV B,R7
                 MOV R5,A;INITIALIZING A FOR FIRST ITERATION
                 ;MOV A,R5 ; BRINGING THE SMALL NUMBER IN A FROM PREVIOUS
ITERATION

NEXT_LOOP_GCD:     MOV R4,B ; PRESERVING THE SMALL NUMBER BEFORE NEXT ITERATION
STARTS
                 DIV AB ;REMIANDER IS IN B
                 MOV A,B ; BRINIGNG THE REMAINDER IN A
```

```
                    CJNE A,#00,NEXT_ITER_FOR_GCD
                    SJMP DONE_FINDING_GCD

NEXT_ITER_FOR_GCD: MOV A,R4 ; RESTORING THE SAVED SMALL NUMBER FROM PREVIOUS
ITERATION
                    SJMP NEXT_LOOP_GCD ; REMAINDER IS STILL IN B WHICH WILL WORK
AS SMALL NUMBER FOR NEXT ITERATION

DONE_FINDING_GCD:  MOV A,R4; SMALL NUMBER FROM LAST ITERATION IS THE GCD
                   MOV R2,A ; STORING THE GCD IN R2 OF REGISTER BANK 2

;FINDING LCM:
                MOV A,R6;RESTORING ORIGINAL NUMBERS FROM STORED REGISTERS
                MOV B,R7
                MUL AB ;ASSUMING THE RESULT IS LESS THAN 255 AND THE LOWER BYTE
IS IN A
                MOV B,R2; FETCHING THE CALCULATED GCD
                DIV AB ;LCM IS IN A
                MOV B,A ; STROING THE LCM IN B

                MOV A,R2 ; STROING THE GCD IN A

                CLR PSW.3; AGAIN GOING BACK TO DEFAULT REGISTER BANK 0
                CLR PSW.4
                RET

;FUNCTION FOR CONVERTING HEX TO DECIMAL
HEX2DEC: MOV B,#0AH ;ASSUMING HEX NUMBER IS IN A
         DIV AB
         MOV R6,B ;LSB IS IN R6
         MOV B,#0AH
         DIV AB
         MOV R5,B
         MOV R4,A ; MSB IS IN R4
         RET

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;LOOK UP
TABLES;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;Q
;ASCII LOOK-UP TABLE FOR EACH ROW
KCODE0: DB '1','2','3','4'     ;ROW 0
KCODE1: DB '5','6','7','8'     ;ROW 1
KCODE2: DB '9','0','(',')'     ;ROW 2
KCODE3: DB '+','-','*','/'     ;ROW 3

PRIME_PROMPT1: DB 'NUMBER FOR PRIME',0
PRIME_PROMPT2: DB 'CHECKING- ',0
YES_PRIME:     DB 'PRIME NUMBER',0
NO_PRIME:      DB 'NOT PRIME NUMBER',0
GCD_PROMPT1:   DB 'NUMBERS FOR GCD/',0
GCD_PROMPT2:   DB 'LCM: ',0
GCD_ANS:       DB 'GCD IS: ',0
LCM_ANS:       DB 'LCM IS: ',0
```

```
SQUARE_DATA: DB 0,1,4,9,16,25,36,49,64,81,100,121,144,169,196,225
CUBE_DATA:   DB 0,1,8,27,64,125,216
END
```

7. Hardware Implementation

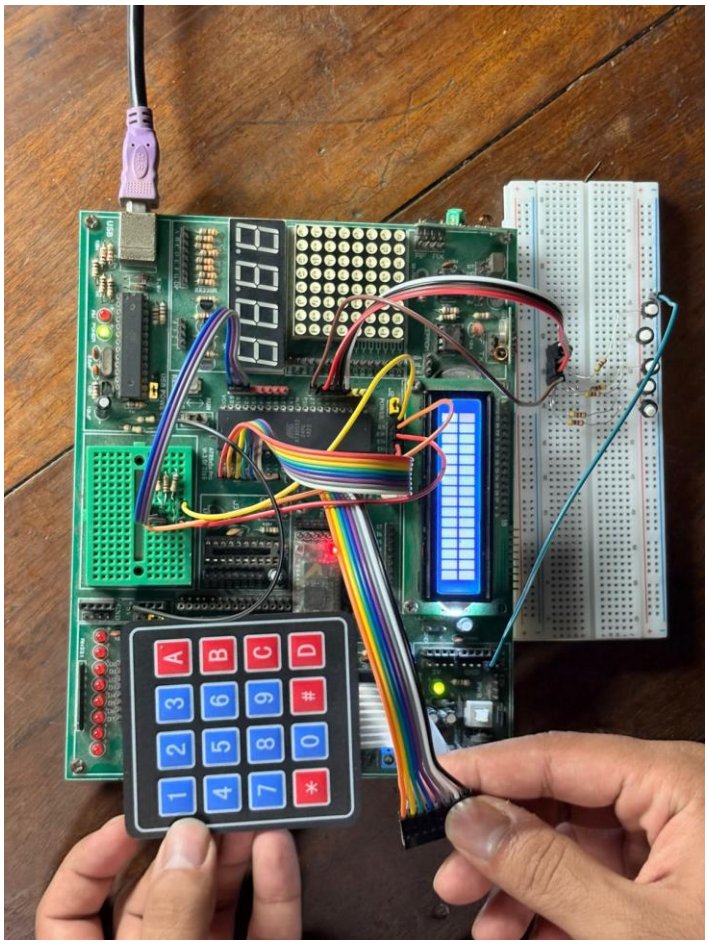


Figure 2: Hardware Implementation of the 8051 Calculator

8. Problems Faced

Problem 1: Jump Address Out of Range with SJMP/AJMP

During early development, SJMP and AJMP instructions were used to branch to subroutines like PRIME_PUSH and GCD_PUSH. As the code grew in size, these addresses moved out of the short/absolute jump range, causing assembler errors.

Solution: Intermediate LJMP (Long Jump) was introduced near the start of the code segment (e.g., POW_PUSH_INTER, PRIME_PUSH_INTER). These stubs use LJMP to reach the actual subroutines, allowing any SJMP or JNB instruction to reach the stub within its limited range.

Problem 2: Register Bank Conflicts in Subroutines

The DELAY subroutine uses R3 and R4 of the current register bank. These registers are also used in the main keypad scanning and expression parsing routines, causing data corruption when DELAY was called mid-computation.

Solution: The DELAY subroutine was modified to explicitly switch to Register Bank 3 (SETB PSW.3, SETB PSW.4) before using R3 and R4, and restore Bank 0 on return. Similarly, PRIME_CHECKER and GCD_LCM_CHECKER use Bank 2 to avoid conflicts with the main code.

Problem 3: Incorrect Number Formation for Multi-Digit Operands

When a user entered a multi-digit number, the RAM buffer stored only the final digit's ASCII value rather than the composed integer value, leading to incorrect evaluation results.

Solution: The FORM_NUMBER routine was implemented to accumulate digits using the formula: current = (previous x 10) + new_digit. R4 holds the running value, and both R4 and the current RAM cell (@R0) are updated on each digit press. When an operator is encountered, R0 is incremented to finalize the number and R4 is cleared for the next operand.

Problem 4: LCD Busy Flag Handling

Without proper busy-flag checking before LCD writes, some commands were missed at higher CPU clock speeds, resulting in garbled display output during rapid key presses.

Solution: A READY subroutine was implemented that polls the LCD's busy flag (P1.7) before each COMNWRT or DATAWRT call. The subroutine drives E high and reads P1.7; if it is HIGH (LCD busy), it loops until the flag goes LOW, ensuring reliable communication.

9. Contribution Table

Student ID	Name	Detailed Tasks
210021107	**Ahmed Fatin Jawad**	Wrote LCD initialization (COMNWRT, DATAWRT, READY subroutines), implemented 4x4 keypad row/column scanning (KEYBOARD_STARTS through FIND/MATCH), and managed keypad ASCII look-up tables (KCODE0–KCODE3)
		Designed and implemented Feature 1: PRIME_PUSH flow and PRIME_CHECKER subroutine (trial division algorithm, register bank switching, carry-flag output, LCD display of "PRIME NUMBER" / "NOT PRIME NUMBER")
		Report writing: Title Page, Objective, Required Components (cost table), and Circuit Diagram sections of the project report, Authored the Features section (mandatory + individual) and the complete Working Principle section including the algorithm flowchart and all subsection explanations
210021139	**Mashrur Saad Emad**	Designed the complete Proteus circuit schematic (AT89C51, LCD, keypad, push buttons, reset circuit)
		Developed the PRECEDENCE function, infix-to-postfix conversion (Shunting-Yard algorithm) within EQUAL_PUSH, and the stack-based postfix evaluator including POW_MODULE with SQUARE_DATA and CUBE_DATA look-up tables
		Implemented Feature 2: GCD_PUSH flow and GCD_LCM_CHECKER subroutine (Euclidean algorithm, LCM formula), and the complete result display pipeline (HEX2DEC, negative number handling via 2's complement, LCD line-2 output)
210021141	**Faisal Abrar Chowdhury**	Implemented multi-digit number formation (FORM_NUMBER), operator/bracket token storage logic (CHECK_1 through CHECK_5), and all arithmetic modules (ADD_MODULE, SUB_MODULE, MUL_MODULE, DIV_MODULE)
		Implemented and assembled the proteus schematic using hardware components; led Proteus simulation testing and debugging of any identified issues
		Report writing: Assembly Code, Hardware Implementation, Problems Faced, and Conclusion sections; led Proteus simulation testing and debugging of all five identified issues
210021149	**Ahmed Awil Mohamoud**	No Contribution
210021157	**Bello Aminu Ado**	No Contribution

10. Conclusion

This project successfully demonstrates the implementation of a feature-rich calculator on an 8051 (AT89C51) microcontroller using pure assembly language. The system correctly performs addition, subtraction, multiplication, division, and exponentiation with proper operator precedence and bracket handling, using the Shunting-Yard algorithm entirely in assembly code. Two additional special features , prime number checking and GCD/LCM computation , expand the utility of the calculator beyond basic arithmetic.

The project provided extensive hands-on experience with low-level hardware interfacing, including LCD initialization and data transfer protocols, matrix keypad scanning, push button debouncing, and register bank management in multi-subroutine assembly programs. Challenges such as jump range limitations, register conflicts, and multi-digit number formation were encountered and resolved. Future enhancements could include support for floating-point arithmetic, larger operand sizes using multi-byte registers, a 4x4 LCD for longer expression display, or porting the design to a more capable microcontroller platform while retaining the assembly-level implementation.